C17 Memory Safety, Pointer Arithmetic & Sanitizers

Object lifetime, bounds, undefined behavior, allocation discipline, and diagnostic tooling

1. C objects, storage duration, and lifetime

C programs manipulate objects with defined storage duration and lifetime. Automatic objects usually live until their block exits; allocated objects live until released; static objects live for the program's duration. Using an object after its lifetime ends, or reading an indeterminate value, can cause undefined behavior.

```
int *bad_pointer(void) {
    int local = 42;
    return &local;  // Wrong: local's lifetime ends on return.
}
```

Design notes: Returning a pointer does not extend the pointed-to object's lifetime. Return a value, accept caller-provided storage, or allocate an object whose ownership and release rule are clear. Compiler warnings and static analysis can help find lifetime mistakes.

Practice: Identify the lifetime and owner of every pointer in a small function that allocates, returns, and frees a buffer.

2. Pointer arithmetic and array bounds

Pointer arithmetic is defined relative to elements of the same array object, including forming a one-past-the-end pointer. The one-past pointer may be used for comparisons and loop termination but must not be dereferenced. Forming pointers outside the permitted range can itself be undefined behavior.

```
int values[] = {10, 20, 30};
int *begin = values;
int *end = values + 3;  // one past the last element

for (int *p = begin; p != end; ++p) {
    printf("%d\n", *p);
}
// Do not read *end.
```

Design notes: Array indexing is defined in terms of pointer arithmetic. An expression such as values[i] is only valid when the resulting element is within the array. Negative indices, off-by-one loops, and integer overflow in index calculations are common hazards.

Practice: Change the loop condition from p != end to p <= end and explain why the body can dereference an invalid one-past pointer.

Additional review: Implementation lens: isolate the smallest reproducible example, state the expected result, and test it independently before integrating it into a larger application.

3. Pointer provenance and type correctness

Pointers are not generic integer addresses for arbitrary arithmetic. Dereferencing requires suitable alignment, a live object, and a compatible effective type. Casting does not make an invalid access valid. Keep pointer operations within the intended object and respect aliasing and alignment rules.

```
unsigned char bytes[sizeof(unsigned int)] = {0};
unsigned int value = 123;
memcpy(bytes, &value, sizeof value);  // bytewise copy is defined
unsigned int copy = 0;
memcpy(&copy, bytes, sizeof copy);
```

Design notes: Use memcpy for representation copying when appropriate rather than dereferencing an incorrectly typed pointer. Character types can inspect object representations, but that does not authorize arbitrary typed accesses. Endianness and representation may vary across platforms.

Practice: Review a cast from char* to int* and list the alignment, bounds, lifetime, and effective-type assumptions it makes.

4. Allocation, overflow, and ownership

Dynamic allocation requires checking failure and ensuring allocation size calculations cannot overflow. A successful malloc returns suitably aligned storage for ordinary object types, but the caller must establish and maintain object contents and free the allocation exactly once.

```
size_t count = /* validated count */;
if (count > SIZE_MAX / sizeof(int)) {
    /* reject size: multiplication would overflow */
}
int *items = malloc(count * sizeof *items);
if (items == NULL && count != 0) {
    /* handle allocation failure */
}
/* use items only within its allocated extent */
free(items);
items = NULL;
```

Design notes: Setting one pointer to NULL after free does not invalidate other aliases. Do not free stack memory, free the same allocation twice, or use a pointer after free. Keep ownership transfer explicit in APIs and document who releases returned memory.

Practice: Add overflow checks to a function that allocates n elements of a structure. Test zero, maximum, and invalid sizes.

5. Buffer length and string safety

C strings are byte arrays terminated by a null character. The destination must have room for both the data and terminator. APIs that receive a pointer without a length cannot independently verify the accessible extent, so interfaces should carry lengths when possible.

```
int copy_text(char *dst, size_t cap, const char *src) {
    if (dst == NULL || src == NULL || cap == 0) return -1;
    size_t n = strlen(src);
    if (n >= cap) return -1;  // need n + 1 bytes
    memcpy(dst, src, n + 1);
    return 0;
}
```

Design notes: This example assumes src points to a valid null-terminated string. If source length is untrusted or input may contain embedded null bytes, use a pointer-plus-length interface. strncpy is not a universal safe replacement because it may fail to terminate.

Practice: Test capacities of zero, one, exactly length, and length plus one. Define whether truncation is allowed or must fail.

Additional review: Review lens: consider edge cases, failure behavior, maintainability, and the assumptions the code makes about its input or execution environment.

6. Undefined behavior and integer arithmetic

Undefined behavior means the C standard imposes no requirements on the program's result. Examples include signed integer overflow, out-of-bounds access, use-after-free, invalid shifts, and dereferencing invalid pointers. Compiler optimization can make bugs behave unexpectedly once undefined behavior is present.

```
int add_checked(int a, int b, int *out) {
    if (out == NULL) return 0;
    if ((b > 0 && a > INT_MAX - b) ||
        (b < 0 && a < INT_MIN - b)) {
        return 0;  // overflow
    }
    *out = a + b;
    return 1;
}
```

Design notes: Include limits.h for INT_MAX and INT_MIN. Unsigned arithmetic wraps modulo its range, but that does not make every unsigned calculation safe: wraparound can still produce undersized allocations or bypass checks.

Practice: Write tests around INT_MIN and INT_MAX for checked addition and multiplication.

7. Defensive API design and const correctness

Use const to express read-only access through a pointer and size_t for object sizes. Validate all public inputs and define failure behavior. A pointer-plus-length pair is usually more auditable than a bare pointer, but the caller's guarantee that the region is valid still matters.

```
int sum_ints(const int *values, size_t length, long long *out) {
    if (out == NULL || (values == NULL && length != 0)) return 0;
    long long total = 0;
    for (size_t i = 0; i < length; ++i) {
        total += values[i];
    }
    *out = total;
    return 1;
}
```

Design notes: The sample assumes the caller provides at least length readable int elements and that the accumulated result fits in long long. Contracts should state these assumptions; checked accumulation may be needed for adversarial or unbounded inputs.

Practice: Write a function contract that describes ownership, nullability, capacity, and failure behavior for a byte buffer.

8. AddressSanitizer and UndefinedBehaviorSanitizer

Sanitizers instrument builds to detect classes of runtime defects. AddressSanitizer is designed to find errors such as heap/stack/global out-of-bounds accesses and use-after-free. UndefinedBehaviorSanitizer detects selected undefined behaviors such as signed overflow and invalid shifts. They complement, not replace, review and tests.

```
# GCC or Clang example build:
cc -std=c17 -O1 -g -Wall -Wextra -Wpedantic \
   -fsanitize=address,undefined -fno-omit-frame-pointer \
   program.c -o program

./program
```

Design notes: Run sanitizers in development and CI with representative tests. Sanitizer builds have overhead and do not prove the absence of bugs; they only detect issues on executed paths and within the checks they implement. Compiler and platform support varies.

Practice: Introduce a one-byte heap overflow in a throwaway test and observe the sanitizer report, then remove the defect.

Additional review: Optimization lens: measure behavior with representative data and workload; check the relevant official documentation before relying on implementation-specific behavior.

9. Static analysis, warnings, and fuzzing

Compile with strong warnings, use static analyzers, and fuzz parsers or APIs that accept untrusted input. Dynamic tests explore concrete executions; static analysis reasons about code paths; fuzzing generates many inputs. Combining methods provides better coverage than relying on one tool.

```
# Example warning-oriented build:
cc -std=c17 -O2 -g -Wall -Wextra -Wpedantic \
   -Wconversion -Wshadow -Wstrict-prototypes \
   program.c -o program
```

Design notes: Warning flags may reveal legacy assumptions and should be introduced systematically. Treat warnings as review prompts, not proof of correctness. Fuzz harnesses should be deterministic enough to reproduce failures and should minimize failing inputs.

Practice: Create a parser that accepts a length-prefixed message and fuzz empty, truncated, oversized, and malformed inputs.

10. Secure coding checklist and review

Memory safety depends on the full chain: valid input, correct length arithmetic, object lifetime, in-bounds access, clear ownership, and reliable error handling. Sanitizers are most useful when the program already has strong tests and repeatable builds.

```
/* Review checklist for a buffer operation:
 * 1. Is pointer nullability specified?
 * 2. Is the accessible length known?
 * 3. Can size arithmetic overflow?
 * 4. Is every index in [0, length)?
 * 5. Does the object outlive every access?
 * 6. Is ownership transferred or released exactly once?
 * 7. Are failure paths safe and testable?
 */
```

Design notes: Prefer established library abstractions where possible, isolate unsafe low-level routines, and keep ownership contracts close to declarations. For C17, consult the C language standard and tool documentation for exact guarantees rather than assuming C++ rules apply.

Practice: Capstone: implement a bounded binary-message parser, run warning-heavy builds and sanitizers, and add tests for malformed input and allocation failure.

Trusted documentation and further reading

• **ISO C draft N1570 (C11 language baseline)** — https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf

• **GCC instrumentation options** — https://gcc.gnu.org/onlinedocs/gcc/Instrumentation-Options.html

• **Clang AddressSanitizer** — https://clang.llvm.org/docs/AddressSanitizer.html

• **Clang UndefinedBehaviorSanitizer** — https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

• **CERT C Coding Standard** — https://wiki.sei.cmu.edu/confluence/display/c

• **SEI CERT C — ARR30-C** — https://wiki.sei.cmu.edu/confluence/display/c/ARR30-C.+Do+not+form+or+use+out-of-bounds+pointers+or+array+subscripts

• **SEI CERT C — MEM31-C** — https://wiki.sei.cmu.edu/confluence/display/c/MEM31-C.+Free+dynamically+allocated+memory+when+no+longer+needed

Subject review checklist

- Pointers refer to live objects and valid bounds.
- Allocation arithmetic is checked for overflow.
- Ownership and release rules are documented.
- Warnings and sanitizers run in CI.
- Malformed and boundary inputs are tested.